SOFTWARE DOCUMENT

Project: ECSE 211 Search & Retrieval Robot

Task: Software design

Document Version Number: 4.4

Date: March 10, 2020

Author: William Bouchard

GitHub link: https://github.com/willbouch/DPMProject

Edit History:

Date	Version	Author(s)	Changes
Feb 25	1.0	William Bouchard	Initial draft
Feb 25	1.1	William Bouchard Andy Li	Reworked the figures, added information on odometer
Mar 10	2.0	William Bouchard	Reworked the Class diagram
Mar 29	3.0	William Bouchard	Corrected a mistake in the software flow
Mar 29	3.1	William Bouchard	Update of section 5.0 to add sketches, a flow diagram and explanations
Mar 29	3.2	William Bouchard	Update of section 6.0 to add sketches, a flow diagram and explanations
Mar 29	3.3	William Bouchard	Update of section 7.0 to add sketches and explanations
Mar 29	3.4	William Bouchard	Added a new section 8.0 to explain how the search on the island would work
Mar 29	3.5	Andy Li	Improved Figures 4, 6, 7, 8
Mar 29	3.6	Andy Li	Improved some explanations
Mar 30	3.7	William Bouchard	Update of section 9.0 to add a flow diagram and explanations
Mar 30	3.8	William Bouchard	Added a new section 10.0
Mar 30	3.9	William Bouchard	Update of section 11.0
Apr 5	4.0	William Bouchard	Update of the class diagram

Apr 5	4.1	William Bouchard	Added section 13.0
Apr 5	4.2	William Bouchard	Added section 14.0
Apr 6	4.3	Andy Li	Revision of the whole document (correction of mistakes)
Apr 6	4.4	William Bouchard	Update of section 14.0

1.0 TABLE OF CONTENTS

2.0 – FLOWCHART **4**

3.0 – CLASS DIAGRAM **5**

4.0 – ODOMETER **6**

5.0 – ULTRASONIC LOCALIZATION **7**

6.0 – LIGHT LOCALIZATION **9**

7.0 – NAVIGATION **11**

8.0 – ISLAND SEARCHING **12**

10.0 – MAIN **16**

11.0 – THREAD ORGANIZATION **17**

12.0 – RELEASES **19**

13.0 – COVID-19 UPDATE **20**
- **13.1 – TODOs** **20**
- **13.2 – IDEAS OF ALGORITHMS** **20**

14.0 – SOFTWARE DESIGN PROCESS **22**
- **14.1 – VERSION CONTROL** **22**
- **14.2 – DEVELOPMENT** **23**
- **14.3 – BUDGET CUTS** **23**

2.0 – FLOWCHART

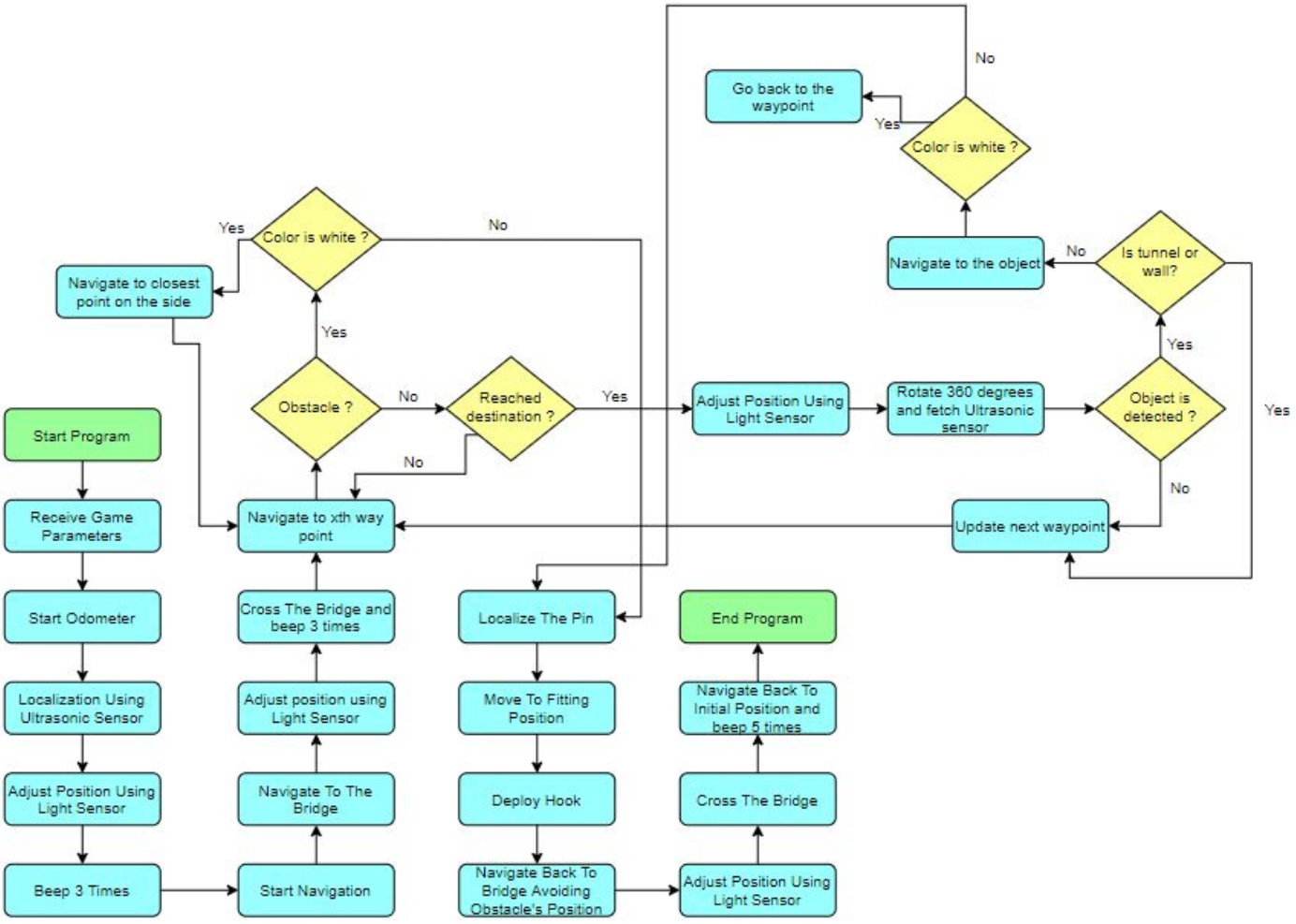


Figure 1: Software flow

3.0 – CLASS DIAGRAM

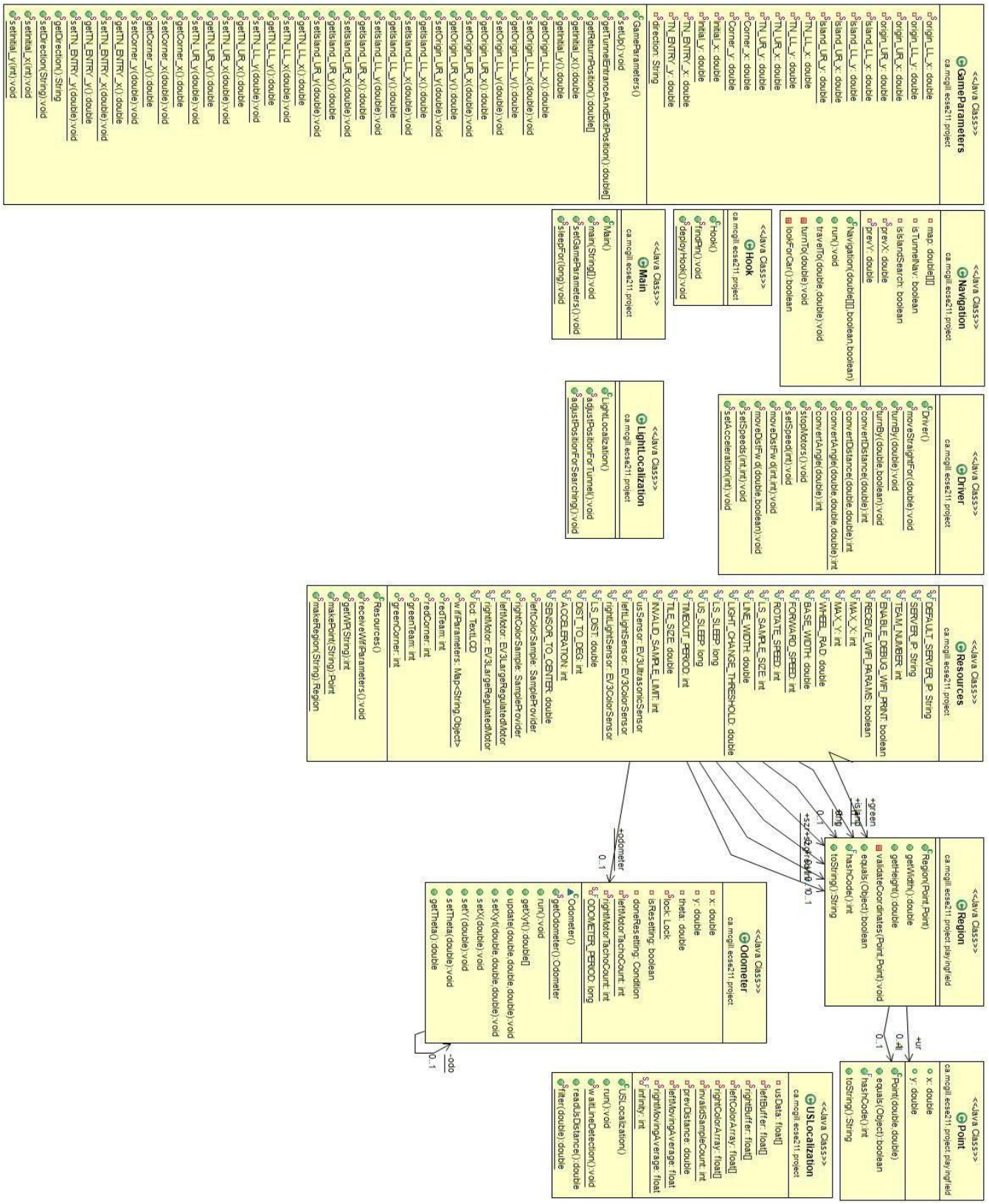


Figure 2: Class diagram

4.0 – ODOMETER

The odometer that we will use is the one that has been developed in Lab 2. We kept the implementation of Andy and William's version of the odometer, since it proved to be very accurate.

The functioning of the odometer in practice is very simple. The odometer class measures the distance that each motor has traveled in terms of degrees rotated. It is then divided to find the number of wheel rotations and multiplied by the circumference (pi times the wheel diameter) to find the linear distance each motor has traveled.

The odometer cannot use this value directly, however, as it must find the X and Y components of the distance traveled. It does so by finding the derivative of each motor's distance traveled by subtracting its cumulative reading from the previous one. It then finds the degree the robot has turned by calculating the difference in distance between the two motors and taking the arctangent of the difference and the wheelbase of the robot. Finally, it finds the X and Y components of the distance traveled by using the sine and cosine functions and integrates the result by adding it to a running total.

The software flow depends on the hardware design, so the software as it is currently written cannot be easily transferred to another robot without significant modification.

5.0 – ULTRASONIC LOCALIZATION

The Ultrasonic (US) localization that we had previously was not accurate enough, so we used mostly new code for this functionality. Figure 3 gives a visual (flowchart) representation of our code's behaviour.

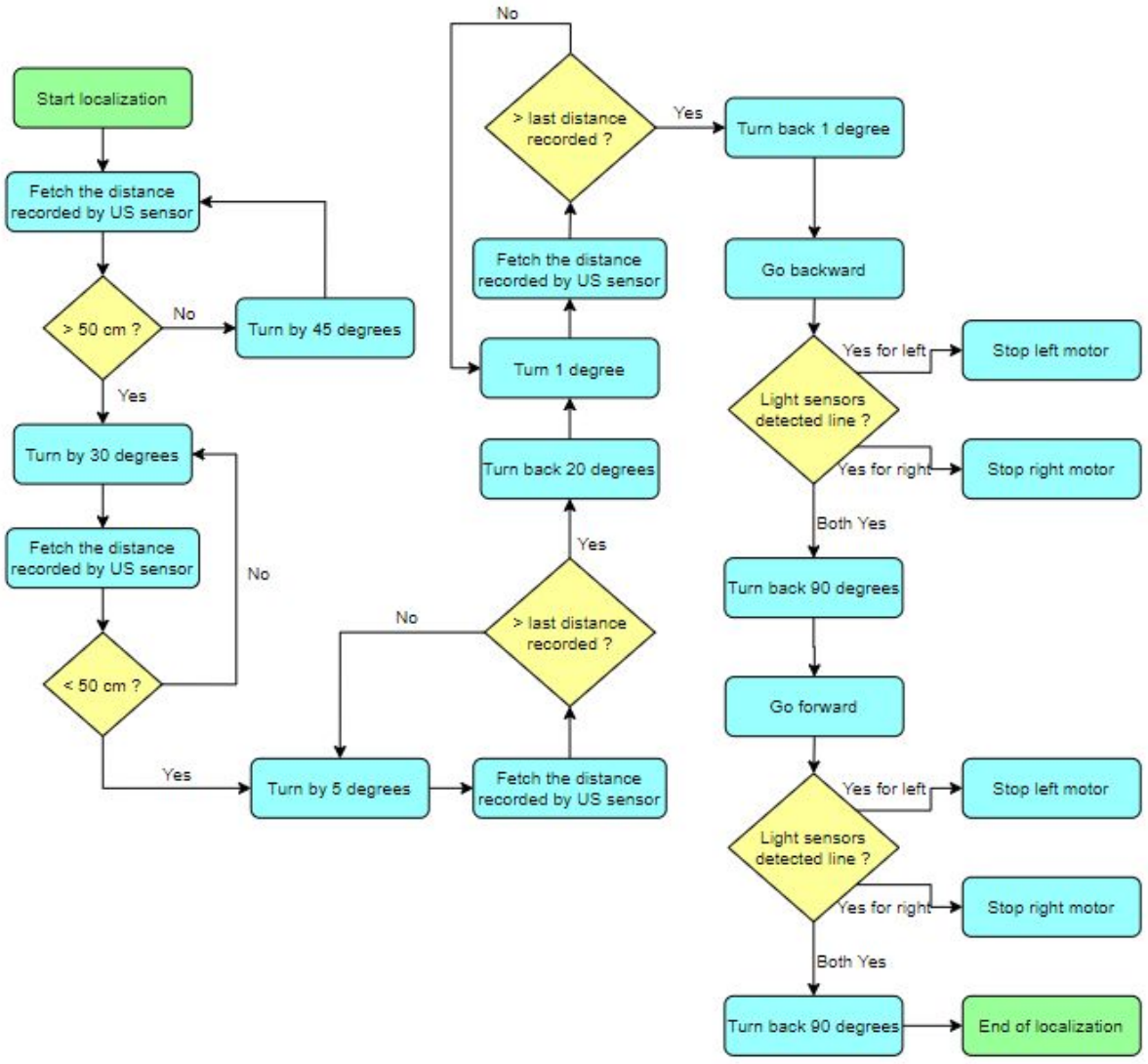


Figure 3: Ultrasonic localization flowchart

The localization starts by turning the robot clockwise until it faces away from the walls (i.e., when the ultrasonic sensor detects a very large distance). Specifically, it turns 45 degrees clockwise and fetches the distance, repeating this process until the measured distance is greater than 50 cm. After that, the robot rotates 30 degrees until it finds a wall (when the fetched distance is less than 50 cm). Then, the robot turns 5 degrees, records the current distance, and compares it to the previous one recorded. When the current distance is greater than the previous one, this means that the robot has reached or moved slightly past the angle at which it is perpendicular to the wall (and thus the closest to it). To increase the precision,

the robot rotates back 20 degrees and does the same comparisons for 1 degree rotations. When it finishes, the robot is almost exactly perpendicular to the wall.

The localizer then corrects its position using its light sensors. For this, the robot moves backward until it detects a line. When the light sensor on one side of the robot detects a line before the other sensor, its respective motor stops; this increases the precision of the localization by rotating the robot slightly around one wheel. Then, the robot rotates back 90 degrees to face the (1,1) point (relative to the wall corner) and moves forward until both sensors detect the line (applying the same angle-correction logic). When it rotates back 90 degrees, it is on the (1,1) point and facing the 0º direction with very high precision.

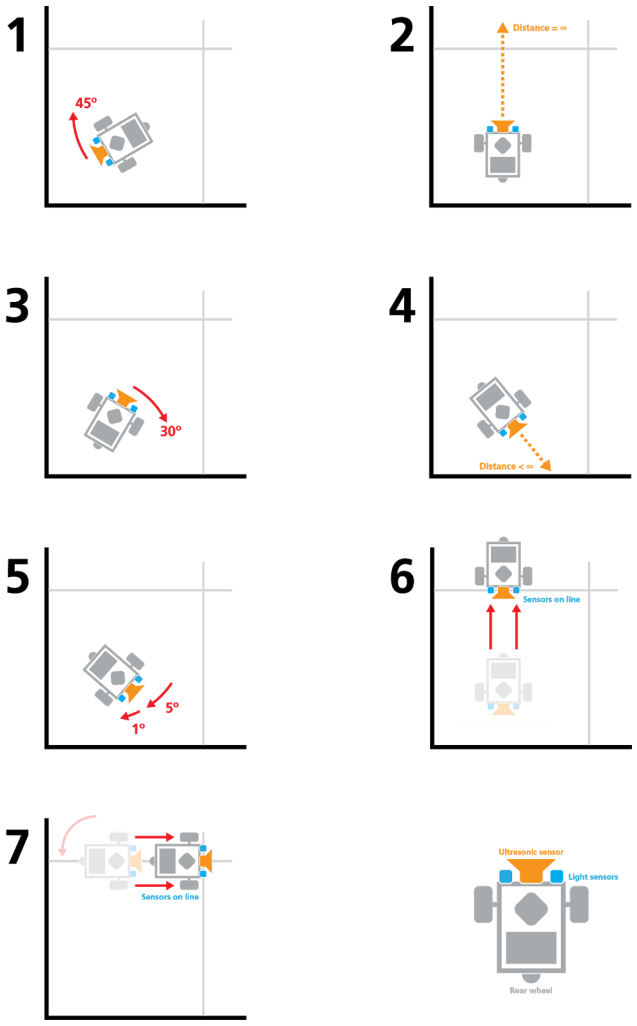


Figure 4. Sketches of US localization

6.0 – LIGHT LOCALIZATION

Like the ultrasonic localizer, the light localizer was redesigned for this project. This localizer is an extension of the one used during the initial ultrasonic localization (Section 5), but with a different purpose, namely for accurate navigation through the tunnel. The behaviour of the localizer is shown in Figure 5.

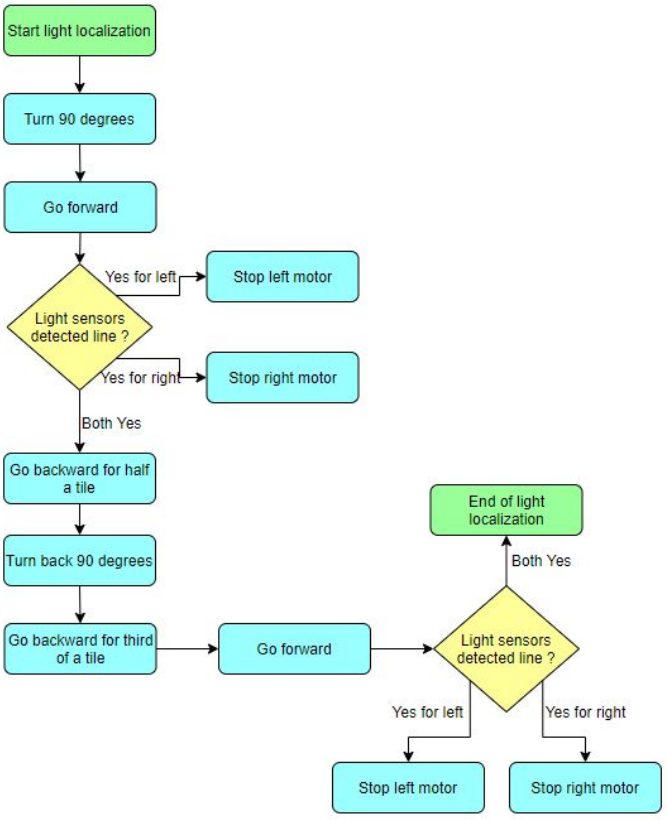


Figure 5: Light localization flowchart

Before entering the tunnel, the robot stops in the middle of the tile closest to the tunnel entrance. To make sure its orientation is exact, the robot turns 90 degrees, corrects its position using its light sensors and the method described in Section 5, reverses half a tile, turns back 90 degrees, and corrects its position for the second axis using the same method. Thus, the robot knows its exact position and angle before entering the tunnel.

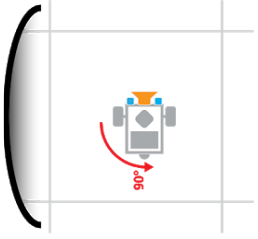


Figure 6: Sketch of light localization behaviour

However, the robot would need to localize once more after passing through the tunnel and onto the “island” section of the course. Although we did not have the time to implement this part, we had a clear idea about how we would have proceeded.

The light localizer’s behaviour would have been similar to the localization system we already developed. Since the odometer keeps track of the robot’s position with high accuracy, we could have found the closest point on the grid after traveling through the tunnel using the Euclidean distance formula; for example, if the robot is supposed to be on the point (6,9), then its X position should be 6 times the (programmed) tile size and its y position should be 9 times this size. The robot would then move to the closest point and localize there using the light sensor method previously developed in the UltrasonicLocalization.

Finally, it is important to note that the localization will also be done when entering the tunnel after the search. However, it will be hard to perform such a localization with the vehicle towed at the back, the robot will disconnect from the trailer, localize, and reconnect. Since the hook is relatively large and the odometer is precise, the error in the position is not expected to be so large that the robot will be unable to tow the vehicle after localization. The claw should be able to compensate for the imprecision in the position before localization and correction.

7.0 – NAVIGATION

The navigation to be used is the one developed in Labs 4 and 5. We will be using Andy and William's version since it provides an accurate calculation of the minimum angle and uses an efficient method to detect when the robot has reached a waypoint.

The navigation is relatively straightforward. It starts by computing the angle at which the next waypoint is located. Using the odometer's current position and the position of the next point, the angle is computed and the robot rotates until it faces the next waypoint.

The robot turns using a minimal angle (never more than 180 degrees) by calculating the relative headings of the initial and target angle. If they vary by 180 degrees or more, the robot will generally turn left; otherwise, it will turn right. This is more complicated if the zero degree axis is between the two angles, as the degree measurement rolls over from 360 to 0 degrees. This can be addressed by adding 360 degrees to the initial angle where appropriate.

Then, the robot starts a while loop to move forward. Every iteration, the robot checks if the destination has been reached yet. If the current Euclidean distance from the waypoint is greater than the last distance, we can safely assume that the robot has reached the point and can exit the loop; otherwise, it keeps moving forward. This method was proven to be very efficient in Labs 4 and 5. Figure 7 gives a visual representation of this principle.



Figure 7. Calculation of stopping position

8.0 – ISLAND SEARCHING

Due to the changes in the project timeline, the code for searching once on the island was not implemented. Before the schedule changes, we had planned but not yet implemented a solution.

Once the robot is on the island, the robot would split the area into four rectangles with about equal area and navigate to the middle of each. At each waypoint, the robot would perform a 360 degree rotation and fetch the ultrasonic sensor to get a distance. If the distance were less than a determined constant, then the robot would note an object in that direction: either a wall, an obstacle, the tunnel, or the car.

Depending on the angle and the (x,y) position of the robot, it would be fairly simple to detect the wall case and the tunnel case, since the robot knows their position from the game parameters. For example, if the robot is at position (1,1) and facing toward a wall, then an object detected at approximately the distance to the wall (calculated using a cosine function) is likely to be the wall itself and can generally be ignored. The same applies to the tunnel, which the robot knows the location of from the game parameters.

The robot has now narrowed down the detected object to either an obstacle or the trailer to which it needs to attach. To know for sure which case it is, we can simply navigate toward the object and use a colour sensor to scan the object. If the obstacle is white, it is an obstacle; the robot can then return to its previous position and continue scanning. In the case of another colour, then it is the trailer; the robot can then begin the process of connecting to the hitch.

Once the trailer is attached, the robot aborts the current navigation and goes back to the tunnel. It then goes through the tunnel to return to the starting point.

At each point, the robot can use the light localization discussed in Section 6 to correct its position. Before going through the tunnel to return to the robot's original island, the localization described and already implemented is used again to ensure the robot is exactly in the middle of the tile facing straight towards the tunnel. Since the trailer can interfere with the robot's motion, which includes reversing and sharp turns, it is detached from the robot at a known position near the localization area and reattached after the robot localizes.

Finally, in the case of an obstacle in the robot's path, it would stop and use the colour sensor to decide if it is facing the trailer or an obstacle. In the case of an obstacle, the robot would record the position of the obstacle to avoid it when returning to the tunnel. It would then select an intermediate point beside the obstacle and navigate to it before navigating to its original destination (Figure 8).

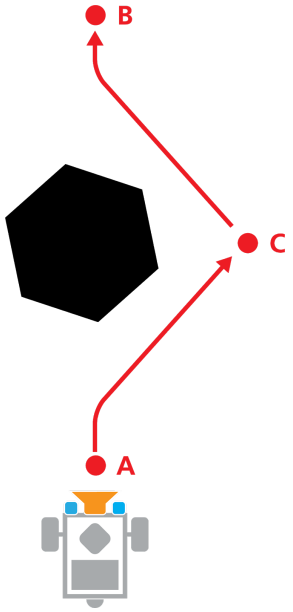


Figure 8: Obstacle avoidance behaviour

9.0 – HOOKING

We were not able to implement this part of the project before the changes in the project timeline, so the following is a description of what we would have implemented if we had the time to do so.

The procedure for connecting the robot to the trailer would have been separated into two methods in our code. The first method would be responsible for finding the pin on the trailer; thus, it would implement a way for the robot to move in a circle around the trailer and use its colour sensor to find the location of the pin at certain points. This search would not have to be extremely precise since the hook itself is large enough to catch the pin even with imprecisions in its location. The second method, which would be responsible for the motor that deploys the hook, would then start. Figure 9 shows the expected behaviour of the robot when connecting to the trailer.

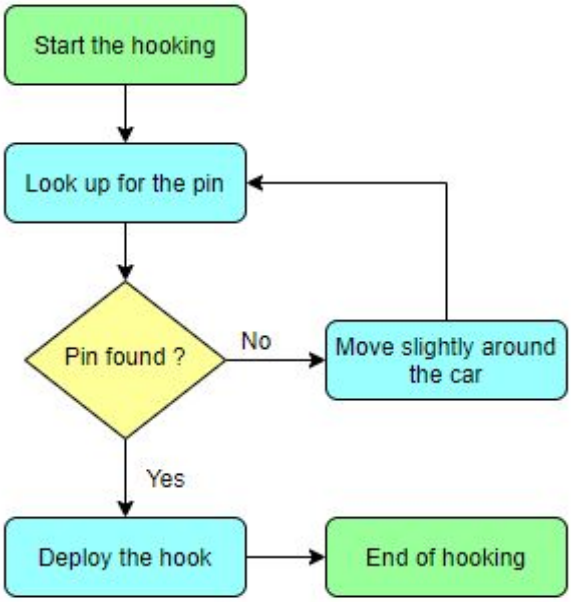


Figure 9: Hooking flow

The procedure is very simple. However, the complicated part resides in the way the robot turns around the car. To address this problem, we came up with two solutions. Obviously, we would have tested both to select the most efficient.

In the first case, we can turn around the car and avoid any collision by using its dimensions. Indeed, the robot could navigate to different waypoints based on the width and length of the car. This method is the simplest one but would need some testing to see if it is realistic.

The second alternative would be to use the ultrasonic sensor and basically implement a bang bang way to turn around the car. This would need further development and testing but would probably be the most efficient method to turn around the car.

10.0 – MAIN

The main method is responsible for the management of threads and execution of all the steps of the software flow. Hence, its flow of execution is the one that can be seen in Figure 1.

11.0 – THREAD ORGANIZATION

During the execution of the whole program, several threads will always be working. Indeed, the Main thread, the odometer and both motors controlling the wheels will be running throughout.

During the initial localization, the UltrasonicLocalization thread will be working, using the ultrasonic sensor in the first part and the two light sensors (located on the sides of the robot) in the second part.

Once the localization is done, the UltrasonicLocalization thread stops and Main starts a new Navigation thread, which is responsible for travelling to the tunnel and reusing the two light sensors to correct the position. Then, this Navigation terminates and a new one is started to cross the tunnel. Again, the Navigation then terminates and a new one is created to manage the search on the island. This was not implemented due to the suspension of the project, but would need to use the ultrasonic sensor to make sure to detect an obstacle if there is one in the way. When an obstacle is found, the colour sensor would then detect the colour and turn off immediately. Thus, when on the island, only the ultrasonic sensor is used during the whole search (though the two light sensors will allow for position adjustment at every waypoint).

Once the car is found and the pin is located, the motor responsible for the hook will be started and turned off immediately afterward; it will not be used again. Then, the robot travels back to the tunnel. At this stage, the ultrasonic sensor is off and not used again. Instead, the two light sensors are used for the last time to adjust the position before the robot enters the tunnel. A new Navigation thread starts when the previous one stops when reaching the tunnel; this one makes the robot cross the tunnel and finally return to the initial position.

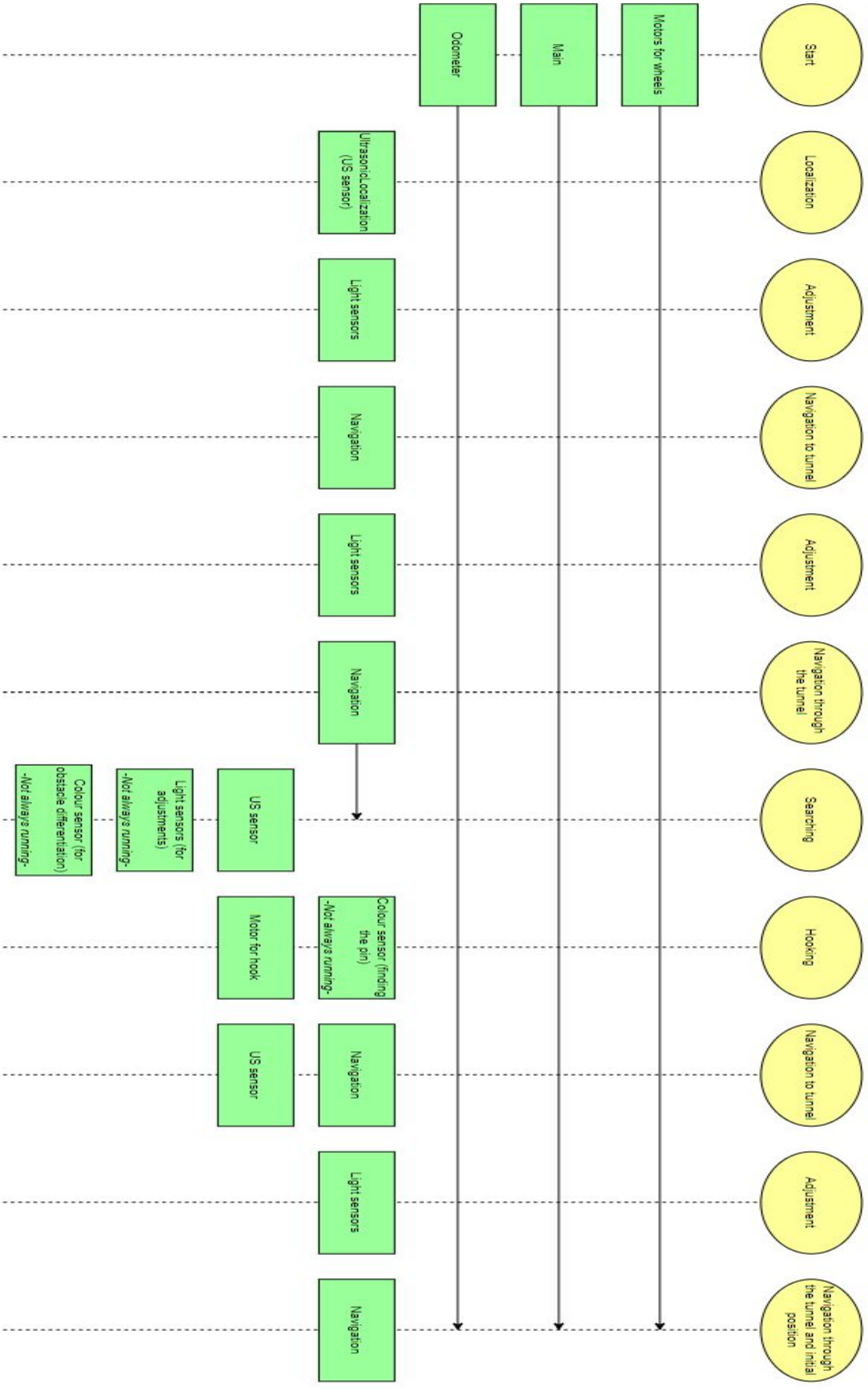


Figure 10. Tread Organization Chart

12.0 – RELEASES

Release version: 1.0

Date: March 3, 2020

Link: https://github.com/willbouch/DPMProject/releases/tag/1.0

Description:

- Implemented basic class hierarchy
- Implemented the main controlling every other thing
- Implemented the core logic of the flowchart
- Reused code from previous labs and assembled them together

Release version: 2.0

Date: March 30, 2020

Link: https://github.com/willbouch/DPMProject/releases/tag/2.0

Description:

- Implemented initial localization
- Implemented navigation to the tunnel
- Implemented light localization correction
- Implemented navigation to cross the tunnel

Release version: 2.0

Date: April 6, 2020

Link: https://github.com/willbouch/DPMProject/releases/tag/3.0

Description:

- Finished the implementation of the main
- Implemented the WiFi parameters reception
- Added modularity to the code by providing multiple new classes
- Left several TODOs in the code for the next team of engineers

13.0 – COVID-19 UPDATE

The budget has been cut significantly due to the COVID-19 crisis. To help out with the transition, this section will list the TODOs in terms of software development and will summarize the design ideas we had imagined that we described in the previous sections. This way, the transition is easier when getting back to work on the project.

13.1 – TODOs

- Hook
    - Implement findPin()
    - Implement deployHook()
- LightLocalization
    - Implement adjustPositionForSearching()
- Navigation
    - Implement lookForCar()
    - Implement the navigation to the car if a car was detected (inside run())
    - Implement the constant Ultrasonic sensor polling to detect obstacles (inside travelTo())
    - Implement the obstacle avoidance (inside travelTo())

The code is also filled with TODOs at the place where each of these points would have to be implemented.

13.2 – IDEAS OF ALGORITHMS

- *Hook findPin() (2 algorithms)*
    - Using the dimensions of the car, estimate different positions around the car. The robot can then navigate to those positions and poll the colour sensor to determine if it found the pin.
    - Using the ultrasonic sensor, implement a bang bang method to mimic wall following and apply it to the car. Then, at regular intervals, poll the colour sensor to determine if it found the pin.
- *Hook deployHook()*
    - This one is straightforward, simply starting the motor responsible for deploying the hook.
- *LightLocalization adjustPositionForSearching()*
    - Reuse the already implemented algorithm to adjust the position of the robot at the end of the initial UltrasonicLocalization with slight modifications. In fact, there would need to be additional logic to

compute the direction in which the robot has to go to adjust its position to the correct point. The idea would be to use theta, x, and y to know approximately where the point on the grid is and then use the two light sensors to adjust the robot to be perpendicular in both directions and finally position the robot exactly on the point (just like at the end of the initial localization).

- *Navigation lookForCar()*
    - This one was well explained and developed in Section 8.
- *Navigation (navigation to car)*
    - This one is also very simple. Simply make the robot go forward until the ultrasonic sensor returns a distance that is smaller than a certain threshold (5 cm is an example)
- *Navigation (obstacle detection)*
    - This one only needs to add an if statement that checks the distance returned by the ultrasonic sensor and compares it with a certain threshold (5 cm is an example). If it is smaller, then an object is right in front of the sensor. Then, use the colour sensor to decide if the object is an obstacle or the car. If it is the car, terminate the Navigation and proceed with the hooking. Otherwise, avoid the obstacle.
- *Navigation (obstacle avoidance)*
    - The idea we had is to compute a new waypoint that would be located on the side of the object. Hence, instead of going from point A to B, we would go from A to C, then C to B and C would be located to one side of the obstacle. This is well explained in Section 8 and can be seen in Figure 8.

14.0 – SOFTWARE DESIGN PROCESS

This section will further explain the way the software was developed and how the change in budget impacted its development.

14.1 – VERSION CONTROL

For version control, Git and Github were used to facilitate the development. The code could be seen and modified by all the team members, making it easy to keep track of the changes made to the code and watch for any conflicts. In other words, it is then easier to detect bugs that were added to the previously existing code since Github allows us to check the changes made to any file in any commit. This would also have been fairly useful if a significant mistake were made, as we could use git to revert any past commits.

Version control was also used to prevent any incident that would have resulted in the loss of the code or parts of it. Every time a team member developed something and was done with it, they could push it to the online repository, which would make it safe from anything that could happen locally on the computer. A good example is Ben's loss of data on his computer in March, which could have resulted in the loss of some of the code he developed.

In addition, GitHub allows us to create releases of the code. This was used to signify when significant updates in the software were made.

GitHub also greatly simplifies code reviews. This functionality was not used because of the early COVID-19 outbreak and project suspension, but we had planned several code reviews after the beta demo to correct some of the bugs we noticed. We had planned to review parts of the code and comment on some of the things that were either inefficient or logically incorrect.

Git also allows us to use branches to work concurrently as efficiently as possible. We had planned to use those after the beta demo since we had imagined that we would have many tasks to complete to get ready for the final presentation. In that case, we would have used different branches to allow any conflicts when working on the same files.

14.2 – DEVELOPMENT

The development of the software worked in several steps:

- *Sketch out the skeleton of the class diagram and how the code is separated*
    - This was to make sure that we had a big picture of how the software would work. It was mainly to target which components of previous labs we would reuse.
- *Selection of previous parts and recycling of those*
    - We selected components of previous labs and started from those. Hence, we had an initial localization that we changed, we also had a navigation that we adapted, etc.
- *Implementation of new parts*
    - We continued by implementing the rest of the code that we needed to develop for the beta demo. Because of the COVID-19 outbreak, we had to cancel the beta demo and slow down the development of the software. Hence, we chose to document the code so that the transition is easy and also leave a clear skeleton (filled with TODOs) to identify exactly where the new code would have been developed (and will be developed by the new team).

14.3 – BUDGET CUTS

The COVID-19 crisis resulted in cuts in the budget allocated for software development. Many hours of work were cut and those remaining had to be done in isolation.

We decided to focus on finishing the development of the parts that we had already started, such as the ultrasonic localization. Since we did not have access to the hardware itself, we decided to implement the skeleton of the software flow that was remaining and leave thorough documentation for everything that needs to be implemented by the future team of engineers. This is why explanations are given inside this document in previous sections, but also by meaningful comments added to the code. This will make the transition easy and efficient as possible.